Statistical Power

Ted Ellsworth, PhD Cole Brookson, MSc

2026-10-15

Free statistical consulting for Yale researchers

Study design · analysis · visualization · writing.

Walk-ins, appointments, and workshops — free for all Yale affiliates.

library.yale.edu/statlab

Introduction

A committee, a reviewer, an IRB, or a funder asks: **how many subjects do you need?**

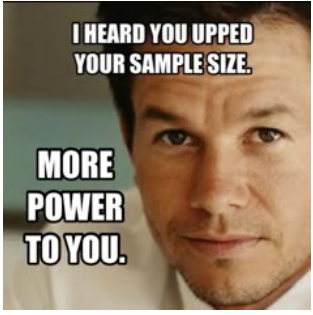


Figure 1: All problems are solved with more power.

Schedule

Leaving this blank until we finalize

Learning Objectives

By the end of the session you should be able to:

1. Describe the formal definition of statistical power, and relate it to other quantities of interest
2. Compute power by hand and with a simple online calculator
3. Understand how to perform a ”sweep” over a grid to identify a sample size
4. Generalize the formal definition of statistical power to an agnostic view of the ”False negative” problem
5. Calculate power from a closed form in the classic hypothesis testing framing
6. Understand and design an agnostic power calculation to quantify effect size recovery in an arbitrary model

False Negatives and False Positives (Type I and Type II Errors)

- Most diagnostic statistics seek to identify two types of errors:
  - *False Positive* (α, Type I) refer to the probability that you will claim an effect is *present* when the **null hypothesis is true**.
  - *False Negatives* (β, Type II) refer to the probability that you will claim an effect is *absent* when the **null hypothesis is false**.
- Power is what is left over from a false negative error: power $= 1 - \beta$.



Figure 2: Dr. Jacob Cohen (1923-1998)

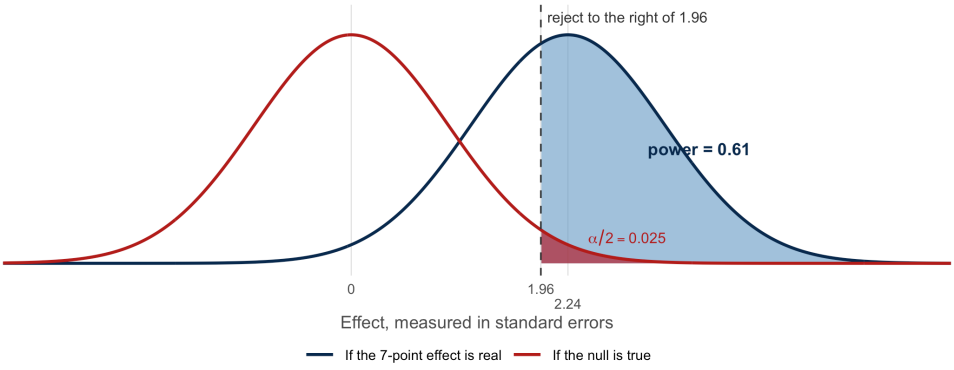
Power is **the probability of rejecting the null hypothesis when it is in fact false and should be rejected.**

Type S and Type M Errors

- Underpowered studies do more than miss real effects: when they *do* find a significant effect, that estimate is likely to be wrong.
- A significant estimate can be wrong in two ways:
  - *Type S (sign)*: the estimate has the opposite sign of the true effect. At 6% power, 24% of significant estimates do.
  - *Type M (magnitude)*: the estimate overstates the true effect. At 6% power, significant estimates are 9.5 times too large on average.

Source: Gelman and Carlin (2014), *Beyond Power Calculations: Assessing Type S (Sign) and Type M (Magnitude) Errors.*

What Power Looks Like



Red is the study you would run if there were no effect; the shaded sliver past 1.96 is α — the false positives you agreed to tolerate. **Blue** is the study you expect if the effect is real; **the shaded area past the same line is power**.

Traditional Statistical Power & Its Ingredients

So what do we need to actually calculate this? Well, the constituent components to calculate $1 - \beta$ are:

- α**, the significance criterion**
- N**, the sample size**
- δ**, the population effect size**

So, you must be thinking, *what's the formula?!*

Traditional Statistical Power & Its Ingredients

There isn't a formula for the calculation of power. There are many!

Core Notation

In order to estimate statistical power, we need four properties:

Symbol	Quality	What it is
N	Sample size	How many subjects you collect in total.
$\mu_t - \mu_c$	Effect size	The true difference between the treatment and control means: the *signal*.
σ	Standard deviation	How much outcomes vary from one another: the *noise*.
α	Significance level	The Type I error rate you are willing to tolerate; it sets the evidence threshold.

Note we will use $\mu_t - \mu_c$ throughout to denote effect size. Where space is tight — in code, and in the appendix — we abbreviate it δ.

The Three Ingredients of Statistical Power

Ingredient	Symbol	Control?	What it does
Strength of the treatment	$\mu_t - \mu_c$	Partly	Dose, framing, and duration are yours to set; a stronger treatment is easier to detect.
Background noise	σ	Partly	Noisier outcomes bury the signal, but better measurement and covariates quiet it.
Experimental design	N, α	**Yes**	Sample size, randomization scheme, and covariate adjustment.

Let's look at a case where we can find a closed form

The formula depends entirely on what *specific* test you want to run. Here's an example. Let's make the following assumptions:

1. We have a completely randomized experiment, with two groups are selected to form a binary treatment, each group containing N 2
2. Assume the outcome variable is normally distributed, and the control group has mean μ_c and the treatment group has μ_t
3. The two groups have a shared standard deviation, σ

We now say we want to perform a difference of means test, in the two-tailed sense

Let's look at a case where we can find a closed form

Say we have the following hypothetical situation where we want to compute power:

- A school is piloting a new **tutoring program**, and wants to know if it improves end-of-term test scores (out of 100)
- Students are randomized into two groups, with one half getting tutoring
- Untutored students average $\mu_c = 72$
- The school will consider the program "worth it", if tutoring gets average scores to $\mu_t = 75$
- Scores vary with $\sigma = 10$

Let's look at a case where we can find a closed form

Say we have the following hypothetical situation where we want to compute power:

- A school is piloting a new **tutoring program**, and wants to know if it improves end-of-term test scores (out of 100)
- Students are randomized into two groups, with one half getting tutoring
- Untutored students average $\mu_c = 72$
- The school will consider the program "worth it", if tutoring gets average scores to $\mu_t = 75$
- Scores vary with $\sigma = 10$

Luckily, in such a simple form, there's a nice asymptotic approximation for the power of the experiment

Step 1: Do the Arithmetic

$$\text{power} = \Phi\left(\frac{|\mu_t - \mu_c|\sqrt{N}}{2\sigma} - \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)\right) \qquad \text{the formula}$$

$$= \Phi\left(\frac{7\sqrt{164}}{2(20)} - 1.96\right) \qquad \text{plug in;} \quad \Phi^{-1}(0.975) = 1.96$$

$$= \Phi(\mathbf{2.24} - 1.96) = \Phi(\mathbf{0.28}) \qquad 7 \times 12.81/40 = 2.24$$

$\mathbf{2.24}$	$\mathbf{1.96}$	$\mathbf{0.28}$
what you expect	what you need	all the room you have

A thin margin means an unreliable study. One lookup turns $\Phi(0.28)$ into a number. (Where does 1.96 come from? Running the z-table *backwards* — see the appendix.)

Step 2: Read the Answer Off the Table

$$\text{power} = \Phi(\mathbf{0.28})$$

Split the z-score across the two margins of the table:

- **0.2** selects the **row**
- **0.08** selects the **column**

The cell where they meet is the area to the *left* of z, which is exactly the power.

z	.00	.01	.02	.03	.04	.05	.06	.07	**.08**	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	**.6103**	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224

$$\text{power} = \Phi(0.28) = \mathbf{0.61}$$

The Smallest Effect This Study Can See

We now know what this design does with a 7-point effect. The next question is what it *can* do: fix power at 80% and solve the same formula for the effect instead of for power.

$$\text{MDE} = \underbrace{\left(\Phi^{-1}\left(1-\tfrac{\alpha}{2}\right)+\Phi^{-1}(0.80)\right)}_{1.96+0.84=2.80} \times \underbrace{\frac{2\sigma}{\sqrt{N}}}_{\text{standard error}}$$

N	Smallest detectable effect	In standard deviations
164	**8.75 points**	**0.44σ**
257	6.99	0.35σ
400	5.60	0.28σ
800	3.96	0.20σ
1396	3.00	0.15σ

Pilot assumes $\mu_t - \mu_c = \mathbf{7}$. This design detects **8.75** and up.

Check: Put 8.75 Back Through the Formula

$$\text{power} = \Phi\left(\frac{8.75\sqrt{164}}{2(20)} - 1.96\right) \qquad \text{the MDE in place of the pilot's effect}$$

$$= \Phi(\mathbf{2.80} - 1.96) = \Phi(\mathbf{0.84}) \qquad 8.75 \times 12.81/40 = 2.80$$

$$= \mathbf{0.80} \qquad \text{row 0.8, column .04} \rightarrow .7995$$

Pin these three	Get
$N = 164, \quad \sigma = 20, \quad \text{effect} = 7$	power = 0.61
$N = 164, \quad \sigma = 20, \quad \text{power} = 0.80$	effect = 8.75

Your Turn: Is the Tutoring Program Worth Studying?

A colleague is evaluating a new tutoring program. **400 students** will be randomly assigned, 200 to tutoring and 200 to business as usual, with final exam score as the outcome. Past cohorts averaged **72** points with a standard deviation of **10**, and the program is expected to add **3** points.

What is the power of this design?

Symbol	Value
N	400
μ_t	75
μ_c	72
σ	10
α	0.05, two-sided

The formula and the z-table are on your **handout**.

Work it in three moves:

1. Compute what you **expect**
2. Subtract what you **need**
3. Look the difference up

Your Turn: The Solution

$$\text{power} = \Phi\left(\frac{|75-72|\sqrt{400}}{2(10)} - \Phi^{-1}\left(1-\tfrac{0.05}{2}\right)\right) \quad \textbf{Step 1:} \text{ plug in}$$

$$= \Phi\left(\frac{3\times 20}{20} - 1.96\right) \quad \textbf{Step 2:} \ \sqrt{400}=20;\ \Phi^{-1}(0.975)=1.96$$

$$= \Phi(3.00-1.96) = \Phi(\mathbf{1.04}) \quad \textbf{Step 3:} \text{ simplify}$$

$$= \mathbf{0.85} \quad \textbf{Step 4:} \text{ row 1.0, column .04} \rightarrow .8508$$

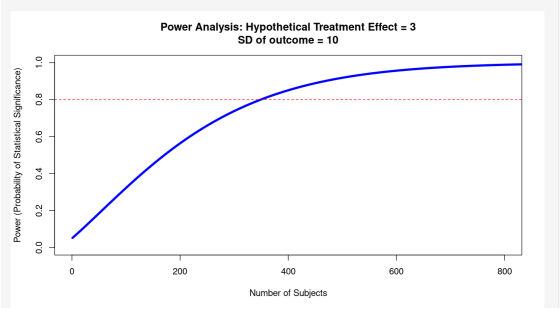
Check Your Work: The EGAP Power Calculator

https://egap.shinyapps.io/power-app/

Our symbol	Field in the app	Enter
$\mu_t - \mu_c$	Treatment Effect Size	3
σ	Standard Deviation of Outcome Variable	10
α	Significance Level	0.05
	Power Target	0.8
	Maximum Number of Subjects	800

Leave *Clustered Design* and *Binary Dependent Variable* unchecked.

Reading the Power Curve



- The x-axis is **total sample size**; the y-axis is **power**.
- The blue curve traces power as N grows, holding the effect at 3 and σ at 10.
- The red dashed line marks the **80% target**. Where the curve crosses it is the N you need (here **349**).
- Our $N = 400$ sits to the *right* of that crossing. Reading up from 400 gives $\approx$ **0.85**.
- The curve **flattens**: past $\sim$600 suggests diminishing returns at larger N.

Where to Situate Power Analysis in Your Research

- Power analysis is tricky because we need information about our data to derive power (e.g., σ, $\mu_t - \mu_c$).
- The problem is power calculations are most useful *before* we've collected our data!
- In order to do power analysis we need to do some educated guessing:
  - How big an effect should I anticipate?
  - How many people can I realistically afford to treat?
  - How many subjects will answer my survey?

What If We Don't Know the Effect?

Back to the survey experiment. The pilot fixed everything *except* for the anticipated effect size.

Fixed by the pilot	
μ_c	50
σ	20
α	0.05
target power	0.80
$\mu_t - \mu_c$	?
N	?

Where an effect could come from	$\mu_t - \mu_c$
Our own pilot	7
A published study of a similar message	5
That published effect, discounted for publication bias	3

The Formula, in Code

$$\text{power} = \Phi\left(\frac{|\mu_t - \mu_c|\sqrt{N}}{2\sigma} - \Phi^{-1}\left(1 - \frac{\alpha}{2}\right)\right)$$

```r
power_calc <- function(mu_t, mu_c, sigma, N, alpha = 0.05) {
    z <- abs(mu_t - mu_c) * sqrt(N) / (2 * sigma)
    cv <- qnorm(1 - alpha / 2)
    pnorm(z - cv) + pnorm(-z - cv) # both rejection tails
}
```

Φ = `pnorm` $\qquad$ Φ^{-1} = `qnorm`

Power Across a Range of N

```r
grid <- expand.grid(N = seq(60, 1600, by = 10), delta = c(3, 5, 7))

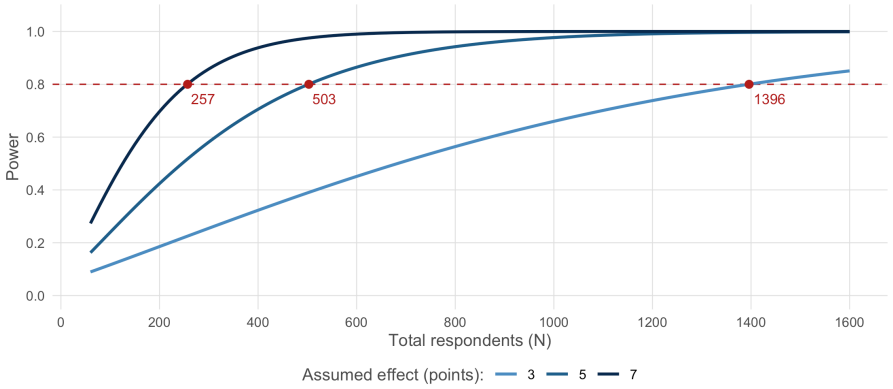
grid$power <- power_calc(
    mu_t = 50 + grid$delta,
    mu_c = 50,
    sigma = 20,
    N = grid$N
)

head(grid, 3)
```

```
   N delta      power
1 60     3 0.08947303
2 70     3 0.09618740
3 80     3 0.10293349
```

One formula, three assumptions, 465 rows. Now plot `power` against `N`.

How Many Respondents Do We Need?



Your Turn: Build the Sweep in R

Your department is considering a six-week **peer-mentoring program** for first-year students, and can randomize students into it or onto a waitlist. The outcome is end-of-year GPA. The registrar tells you first-year GPA averages **3.05**, with a standard deviation of **0.45**.

Given	
μ_c	3.05
σ	0.45
α	0.05

You decide:

- Which GPA gains are plausible?
- How many students could you enroll?

Your Turn: One Solution

```
grid <- expand.grid(N = seq(50, 1000, by = 10), delta = c(0.10, 0.15, 0.20))
grid$power <- power_calc(
    mu_t = 3.05 + grid$delta,
    mu_c = 3.05,
    sigma = 0.45,
    N = grid$N
)

for (d in unique(grid$delta)) {
    ok <- grid$N[grid$delta == d & grid$power >= 0.80]
    cat("effect", d, "GPA points -> N =", min(ok), "\n")
}
```

```
effect 0.1 GPA points -> N = 640
effect 0.15 GPA points -> N = 290
effect 0.2 GPA points -> N = 160
```

Your effects and N range will differ. The *structure* is the answer: one grid, one call, one threshold.

But what about literally any other case?

There are lots of reasons why, even for simple problems, we don't want to rely on such an approximation. For instance, what if you

1. don't assume a two-tailed test?
2. have clustering and want some sort of random effect?
3. are taking a Bayesian approach?
4. are using nonparametric statistics?

What should we do in those other cases?

- Even when power is not as clearly defined, there are still a whole range of things we still want to know about "power"
- Let's start relaxing our definition of power. We can, for now, consider it as:

The probability that we we recover a real effect, should one exist

What should we do in those other cases?

What would it mean then, to approach asking questions about power or sample size in a more general sense where we may want to consider:

- Non-parametric approaches
- Bayesian approaches
- Or even just moderately more complicated study designs?

Well, we go gambling



Figure 3: Oxford Casino Hotel

An aside into the casinos at Monaco

- Monte Carlo refers to a broad class of methods wherein you use repeated random sampling to obtain some form of numerical result
- It will be very useful for us when we cannot rely on a closed form solution (which, as a reminder, is in most cases!)
- Essentially, we can use randomness to solve deterministic problems

For any problem with probabilistic interpretation, it can be solved with Monte Carlo



Figure 4: Primary developer of the MC method, Stanisław Ulam, who was inspired by his uncle's gambling

An aside into the casinos at Monaco

Let's imagine we want to know the answer to a gambling-related problem with a closed-form answer:

What's the probability you win on red at the roulette table three spins in a row?

- There are 18 red pockets out of 38, so the exact answer is

$$P(\text{3 reds in a row}) = (18/38)^3 \approx 0.106$$



Figure 5: A reasonably standard roulette table

An aside to the casinos at Monaco

But we could also just... go to the casino 100,000 times:

```
set.seed(2026)
spins <- matrix(runif(3 * 100000) < 18 / 38, ncol = 3) # each row is 3 spins
mean(rowSums(spins) == 3) # proportion of rows where all three won
```

```
[1] 0.1059
```

If we can describe how the data are generated, we can estimate *any* probability by running the process many times!

An aside into the casinos at Monaco

And whether its the more traditional, rigid form, or our more flexible modern form, power is *just a probability.*

So, how do we do a Monte Carlo simulation, anyways?

In the parametric context, a Monte Carlo power analysis looks like this:

1. **Generate** random data from some distribution with the properties that you expect your real data to have
2. **Analyze** this randomly generated data set with your statistical analysis, whatever that is
3. **Store** the value(s) of the analysis that you're interested in
4. **Repeat** the above steps many times
5. **Compute** power by looking at the proportion of times the statistic or value you're interested in was "correct"

This can be arbitrarily complex or simple!

So, how do we do a Monte Carlo simulation, anyways?

We can consider all of this to be done, for the sake of simplicity, by three piecemeal functions that we will want to write

- `generate()`: make a fake data set that looks like your study
- `analyze()`: run the analysis you'd *actually* run
- `success()`: decide whether this run "recovered" the effect

and one function that necessarily performs the actual simulation

```
simulate_power <- function(n_sims, generate, analyze, success) {
    hits <- replicate(n_sims, success(analyze(generate())))
    mean(hits) # proportion of successful runs = power
}
```

So, how do we do a Monte Carlo simulation, anyways?

We need to make a statement on how we think the data are being generated. For student i:

$$\text{post}_i = \underbrace{\mu_c}_{\text{baseline}} + \underbrace{\delta\, T_i}_{\text{tutoring effect}} + \underbrace{\varepsilon_i}_{\text{noise}}, \qquad \varepsilon_i \sim N(0, \sigma^2)$$

- $\mu_c = 72$: the average score *without* tutoring
- T_i: 1 if student i was tutored, 0 if not, so tutored students get an "effect" of $\delta = 3$
- ε_i: our random noise with $\sigma = 10$

So, how do we do a Monte Carlo simulation, anyways?

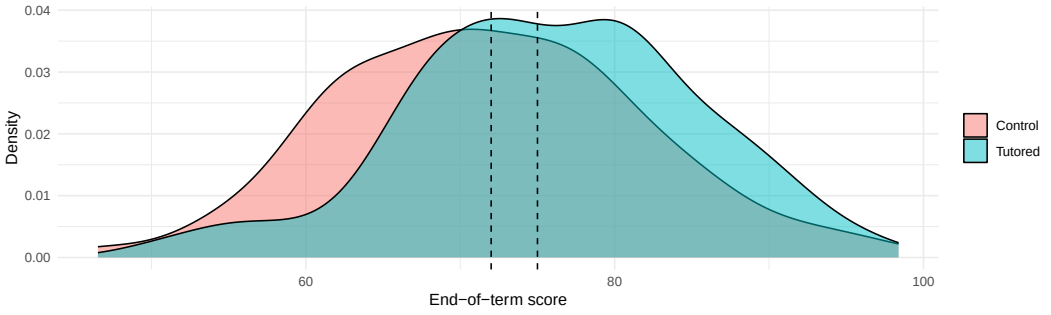
Let's start with our function that generates data based on our student tutoring example from before.

```
gen_basic <- function(n = 350, effect = 3, sd = 10) {
    treat <- rep(0:1, each = n / 2) # half control, half tutored
    post <- 72 + effect * treat + rnorm(n, mean = 0, sd = sd)
    data.frame(treat, post)
}

head(gen_basic())
```

```
  treat     post
1     0 60.03729
2     0 86.41172
3     0 82.52832
4     0 66.93240
5     0 74.34064
6     0 72.20875
```

So, how do we do a Monte Carlo simulation, anyways?



So, how do we do a Monte Carlo simulation, anyways?

```r
analyze_lm <- function(dat) {
  fit <- lm(post ~ treat, data = dat) # same as an equal-variance t-test
  summary(fit)$coefficients["treat", ] # estimate, SE, t, p for treat
}

success_sig <- function(est) {
  est[["Pr(>|t|)"]] < 0.05 # did we reject the null?
}

analyze_lm(gen_basic())
```

```
  Estimate Std. Error    t value   Pr(>|t|)
2.48850940 1.10064783 2.26094971 0.02437943
```

Does the casino agree with our nice equation??

```
set.seed(2026)
simulate_power(
    n_sims = 2000,
    generate = gen_basic,
    analyze = analyze_lm,
    success = success_sig
)
```

```
[1] 0.8105
```

- Pretty close for only 2000 simulations!

Does the casino agree with our nice equation??

- It's worth noting that the simulation has its own error! With S simulations, the Monte Carlo standard error is

$$\text{SE} = \sqrt{\frac{p(1-p)}{S}}$$

Want a more precise answer? **Go to the casino more times.**

What if we make this more complicated, arbitrarily so?

I suspect that schools know a lot about their students *before* the program starts. Say we also have each student's **pre-test score**. Our model for student i becomes:

$$\text{pre}_i \sim N(70, \sigma^2)$$

$$\text{post}_i = \underbrace{\mu_c}_{\text{baseline}} + \underbrace{\rho\,(\text{pre}_i - 70)}_{\text{what we knew already}} + \underbrace{\delta\, T_i}_{\text{tutoring effect}} + \underbrace{\varepsilon_i}_{\text{noise}}, \qquad \varepsilon_i \sim N(0,\ \sigma^2(1-\rho^2))$$

- $\rho = 0.6$ is the correlation between pre-test and end-of-term scores
- Students who were above average before tend to stay above average
- The total spread of scores is still $\sigma = 10$

What if we make this more complicated, arbitrarily so?

- We only need to change **two** of our three pieces:

```
gen_pretest <- function(n = 350, effect = 3, sd = 10, rho = 0.6) {
  treat <- rep(0:1, each = n / 2)
  pre <- rnorm(n, mean = 70, sd = sd)
  post <- 72 +
    rho * (pre - 70) +
    effect * treat +
    rnorm(n, mean = 0, sd = sd * sqrt(1 - rho^2))
  data.frame(treat, pre, post)
}

analyze_ancova <- function(dat) {
  summary(lm(post ~ treat + pre, data = dat))$coefficients["treat", ]
}
```

What if we make this more complicated, arbitrarily so?

```
set.seed(2026)
simulate_power(2000, gen_pretest, analyze_lm, success_sig)
```

```
[1] 0.807
```

```
simulate_power(2000, gen_pretest, analyze_ancova, success_sig)
```

```
[1] 0.9415
```

- We can see that our power jumps from about 80% to about **94%**
- Power is a property of the design *and* the analysis, not just of N
- If we were really strapped for students to enroll, this could be really helpful to know!

(To be fair: this case still has a closed form (σ just becomes $\sigma\sqrt{1-\rho^2}$) but we didn't need to know that!)

Let's make it even more complicated!

Consider the classic problem of dropout. Let's make the following assumptions about our study (maybe we base this on some sort of past experience running such a study):

- Students who are doing *badly* are more likely to stop showing up
- The **control group loses more students** (here, about 23% vs. 11%)

You may ask: *Is there a formula for the power of this study?*

Not really. Dropout depends on an *unrealized outcome*, so the sample size, the effect we estimate, *and* the noise all change at once!! What a pain!!

Let's make it even more complicated!

But we don't need a formula! We can use our computers!! We just need to describe how the data are generated:

```
gen_dropout <- function(n = 350, effect = 3, sd = 10, rho = 0.6) {
    dat <- gen_pretest(n, effect, sd, rho)
    p_drop <- plogis(-2.2 + 0.7 * (dat$treat == 0) - 0.12 * (dat$post - 72))
    dat[runif(n) > p_drop, ] # we only observe the students who stayed
}

nrow(gen_dropout()) # out of the 350 we enrolled
```

```
[1] 289
```

Let's make it even more complicated!

```
set.seed(2026)
simulate_power(2000, gen_pretest, analyze_ancova, success_sig)
```

```
[1] 0.9425
```

```
simulate_power(2000, gen_dropout, analyze_ancova, success_sig)
```

```
[1] 0.6815
```

- Remember, dropout doesn't just shrink our sample!

Let's make it even more complicated!

Because we simulated the data, we *know* the true effect is 3. So we can check:

```
set.seed(2026)
estimates <- replicate(2000, analyze_ancova(gen_dropout())[["Estimate"]])
mean(estimates)
```

```
[1] 2.229586
```

- Since students doing poorly *leave*, this biases our results for the control group up
- We appear to **underestimate** the effect

Unfortuantely, there's no formula for this!!

Let's make it even more complicated!

Remember our relaxed definition: **the probability that we recover a real effect, should one exist**.

- "Recover" doesn't have to mean a p-value.

```
success_close <- function(est) abs(est[["Estimate"]] - 3) < 1.5

set.seed(2026)
simulate_power(2000, gen_pretest, analyze_ancova, success_close) # no dropout
```

```
[1] 0.9185
```

```
simulate_power(2000, gen_dropout, analyze_ancova, success_close) # dropout
```

```
[1] 0.786
```

Let's make it even more complicated!

There are other options for recovery! You could ask when:

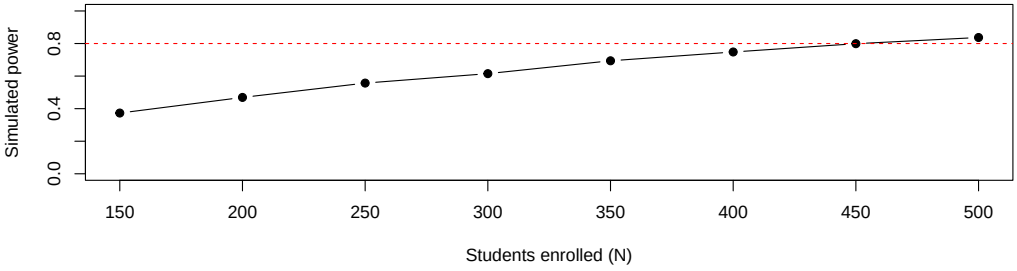
- the CI excludes zero
- the CI is narrower than some width
- the sign is right
- a posterior probability exceeds 0.9 or 0.95
- and much more!

So, how many students do we need?!

Just like before, we want to find the N that gets us to 80%. Now, instead of inverting a formula, we try a grid of sample sizes:

```
set.seed(2026)
ns <- seq(150, 500, by = 50)
powers <- sapply(ns, \(n) {
  simulate_power(
    1000,
    generate = \() gen_dropout(n = n),
    analyze = analyze_ancova,
    success = success_sig
  )
})
```

So, how many students do we need?



With dropout, we need to enrol **around 450 students**. Alas, more students only buys *power* because the estimate is still biased.

Your turn!

We provided you with a template file to use for htis part, `power-sim-template.R`. Use this and fill in every `___`. Use your own study, or, if nothing comes to mind, the tutoring study from earlier is good practice too!

Ask yourself:

1. What are my baseline, effect, and noise values, and where did I get those numbers?
2. `generate()` – Does this dataset look like something my study could produce?
3. `analyze()` – What would I *actually* run on the real data?
4. `success()` – What counts as "recovering" the effect?
5. Estimate power – With *no* effect, do I get about 5%? A significance-based `success()` should succeed about 5% of the time
6. Perform a sweep over N – Where does my curve cross 80%?
7. Now, **add a complication** – Maybe unequal groups, a covariate, a skewed outcome, with these how much power do I lose?